



ACCUMULATORS

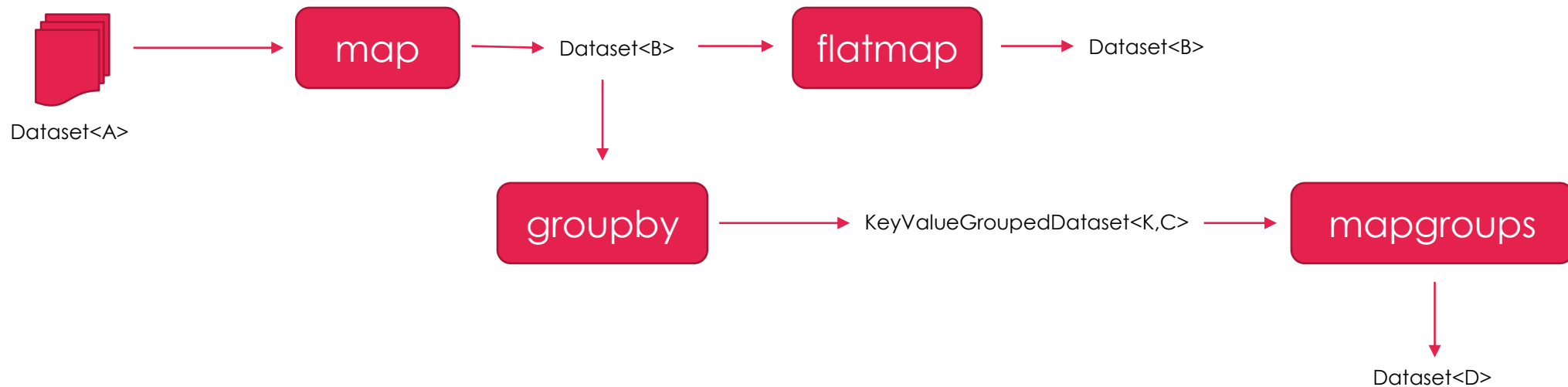
In Spark

Big Data H/M 2022

Richard McCreadie

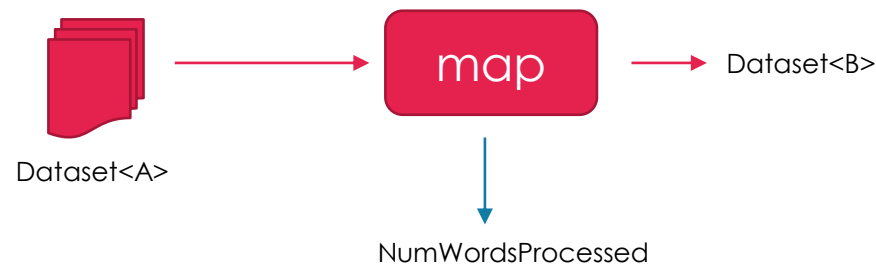
MOTIVATION

- In general, we can consider a spark application as a topology of dataset transformers that convert our data from one form to another, e.g.



MOTIVATION

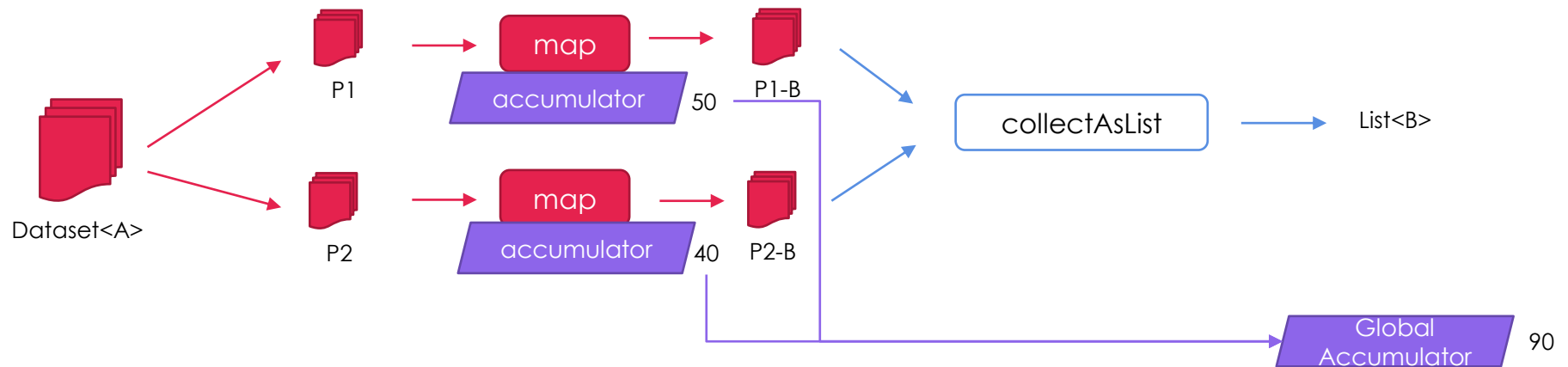
- However, there are cases where during a transformation function we want to also collect additional information
 - E.g. statistics about the items being processed



- This use-case is solved using [Accumulators](#)

ACCUMULATORS

- You can think of accumulators as global counters that you can pass to transformation functions and then increment within those functions
 - At the end of a task, any local accumulators will be aggregated with the global accumulator



ACCUMULATORS

- Accumulators can be created using the spark context, similarly to broadcast variables
 - Accumulators can be made for Long or Double values (effectively acting as global counters)
 - A collection accumulator is also supported, which can accumulate Java objects (so long as they are serializable)

```
LongAccumulator myAccumulator = spark.sparkContext().longAccumulator();
```

- You can then pass the accumulator as a variable to any of your transformation functions, and later update it during calls while that function is in operation

ACCUMULATORS

- In the driver program you can request the value of your accumulator using the `value()` method

```
myAccumulator.value()
```

- However, `value()` is not a spark action, meaning it will not trigger computation
 - This means that you need to make sure there is an action between the time you call your transformation function and request the value